

BCG X Data Science – CodeSignal

Cours a memoriser – fiche d'assessment (90 min, Python)

A imprimer. Les 4 domaines : Data Cleaning / Data Querying / EDA & Stats / Machine Learning. Reflexe attendu : lire l'enonce, reconnaitre le patron en 10 s, taper l'idiome de memoire.

0. Reflexe d'ouverture – scan 30 s

Toujours faire ces 6 lignes sur chaque table inconnue :

```
df.shape           # taille -> perf ?
df.head(3)         # que vaut 1 ligne ? la granularite ?
df.dtypes          # dates/nombres en object = alerte
df.isna().sum()    # ou sont les trous ?
df.nunique()       # "numerique" qui est en fait
                  # categoriel ?
df.describe(include="all") # min/max impossibles ?
```

4 methodes de raisonnement (a reciter) :

CLEANING : inspecter → schema → valeurs manquantes → valeurs invalides → types → doublons → valider (assert).

QUERYING : sortie voulue ? → quelles lignes ? → quelles colonnes ? → agregation ? → jointure ? → cle de groupe ? → tri + top N ?

EDA : cible ? → distributions ? → manquants ? → outliers ? → relations feature/cible ? → correlation ? → interpretation metier.

ML : X ? → y ? → classif/regression ? → split AVANT tout fit → fuite ? → preprocessing dans un pipeline → modele → metrique (cout FP vs FN) → validation croisee → interpretation.

1. Pandas – nettoyage

Types

```
pd.to_datetime(s, errors="coerce") # invalide -> NaT
pd.to_numeric(s, errors="coerce")  # invalide -> NaN
pd.to_numeric(s.astype(str).str.replace(r"[$,]", "", regex=True),
              errors="coerce")     # "$1,200" -> 1200.0
s.astype("category"); s.astype("Int64") # Int64 = entier
nullable
s.str.strip().str.lower().str.replace(" ", "_")
s.str.strip().str.lower().map(MAPPING)  # categories :
synonymes
```

astype(int) avec des NaN → erreur. Rester float ou "Int64". to_datetime sans coerce sur "" → plante. Toujours value_counts() apres un map de categories.

Valeurs manquantes

```
df.isna().mean().mul(100).round(1) # % manquant par colonne
df.dropna(subset=["id"])           # jeter si cle absente
s.fillna(s.median())              # asymetrique/outliers ->
mediane
s.fillna(df.groupby("k")["s"].transform("median")) # fill par
groupe
# valeur impossible -> NaN AVANT d'imputer :
df.loc[(df.age<=0)|(df.age>110), "age"] = np.nan
```

Mediane vs moyenne : distribution asymetrique / outliers (revenu, prix, comptes, durees) → **mediane**. Symetrique → moyenne. Asymetrie a droite ⇒ moyenne > mediane.

Cle primaire manquante : on jette la ligne (on n'invente jamais un id). Justifier le choix d'imputation = ce qui est note.

Doublons

```
df.duplicated().sum()           # doublons exacts
df.drop_duplicates()           # exacts d'abord
df.drop_duplicates(subset=["id"], keep="last") # doublon de cle
metier
assert df["id"].is_unique
```

- **Doublon exact** : toutes colonnes identiques – toujours sur a jeter.
- **Doublon de cle metier** : l'id se repete, le reste differe – choisir une regle (keep="last", plus complet, plus recent).
- **Cle primaire** : garantie d'unicite + non-null – on l'assert.

2. Pandas – interrogation

Filtrer

```
df[(df.a > 1) & (df.b < 10)] # &, |, ~ ; parentheser
chaque test
df[df.c.isin(["x", "y"])]   # pas de and/or sur des Series
df[~df.c.isin(bad_set)]    # "pas dans un ensemble"
df[df.x.between(lo, hi)]   # inclusif des deux cotes
df.query("status=='completed' and 100<=amount<=300")
top10 = df.product_id.value_counts().head(10).index # top-10
categories
```

df[df.a>1 & df.b<2] est lu df.a > (1&df.b) < 2. Parentheser. and/or sur une Series → "ambiguous truth value". Filtrer un pays avant de normaliser le texte → resultats faux. Comparer des dates encore en str → comparaison lexicale.

GroupBy

```
df.groupby("k")["v"].mean() # .sum() .size() .count() .nunique()
.median()
df.groupby("k").agg(n=("v", "size"), total=("v", "sum"),
                  avg=("v", "mean"), ncust=("cid", "nunique"))
df.groupby("k")["is_fail"].mean() # TAUX = moyenne d'un
boolean
df.groupby("k")["v"].transform("mean") # diffuse l'agregat sur
les lignes
s.sort_values(ascending=False).head(N) # top N
```

size = lignes du groupe (compte NaN). count = valeurs non-nulles. nunique = valeurs distinctes.

Un filtre sur une metrique de groupe (n_orders>=4) se fait apres le groupby.

Jointures

```
a.merge(b, on="k", how="left") # left / inner /
right / outer
a.merge(b, on="k", how="left", indicator=True)
# -> colonne _merge : both / left_only / right_only ; audit :
m["_merge"].value_counts()
```

Regle : left pour enrichir la table d'analyse (on garde toutes les lignes). inner si une ligne n'a aucun sens sans son correspondant (vente d'un produit absent du catalogue).

Pieges : types de cle differentes (→ 0 correspondance silencieuse) ; table de droite non dedupliquee (→ explosion 1-vers-N et toutes les sommes fausses) ; oublier suffixes=.

Dates

```
s.dt.year / .month / .day / .weekday # lundi = 0
s.dt.to_period("M")                  # index 2022-01, 2022-02,
...
df.groupby(pd.Grouper(key="d", freq="MS"))["v"].sum() # MS/W/D/QS
/YS
(df.end - df.start).dt.days          # duree en jours
cutoff = df.d.max() - pd.Timedelta(days=30) # "30 derniers jours"
.dt ne marche que sur du datetime reel – convertir d'abord.
```

Feature engineering (1 ligne par entite)

```
t["amt_ok"] = t.amount.where(t.status.eq("completed")) # sinon
NaN
t["is_fail"] = t.status.eq("failed")
feat = t.groupby("customer_id").agg(
```

```
total=("amt_ok","sum"), avg=("amt_ok","mean"),
n=("transaction_id","size"), fail_rate=("is_fail","mean"),
first=("date","min"), last=("date","max"),
n_prod=("product_id","nunique"))
feat["lifetime_days"] = (feat.last - feat.first).dt.days
```

Series.where(cond) garde la valeur si vrai, NaN sinon;
sum et mean ignorent NaN → total/moyenne "completed"
gratuits.

3. Les 5 patrons recurrents

1. **Top N entites par un agregat**
df.groupby(k)[v].agg(f).sort_values(ascending=False).nlargest(N)
2. **Taux / part d'une condition**
df.groupby(k)[bool_col].mean()
3. **Enrichir une table de faits**
faits.merge(dim, on=k, how="left")
4. **Garder les lignes presentes dans un referentiel**
df[df[k].isin(autre[k])]
5. **1 ligne par entite depuis un journal d'evenements**
events.groupby(k).agg(...)

Squelette du *mini-challenge* : nettoyer → masque de validite (1 expression parenthesee) → groupby(entite).agg → derives (ratios/differences) → filtres de seuil → merge dims → sort_values(metrique).head(N) → lancer les assert.

```
assert utiles : df["id"].is_unique;
s.between(0,1).all(); out.shape[0]<=N;
s.is_monotonic_decreasing; not
out.isna().any().any().
```

4. Statistiques

```
s.mean(); s.median(); s.mode()
s.var(ddof=1); s.std(ddof=1) # ddof=1 = echantillon (n-1)
s.quantile([.1,.25,.5,.75,.9])
IQR = s.quantile(.75) - s.quantile(.25)
s.skew() # >0 queue a droite
```

Outliers

Regle IQR (sans hypothese de loi – choix par default) :

$$\text{bornes} = [Q_1 - 1.5 \text{ IQR}, Q_3 + 1.5 \text{ IQR}]$$

```
q1,q3 = s.quantile([.25,.75]); iqr = q3-q1
lo,hi = q1-1.5*iqr, q3+1.5*iqr
((s<lo)|(s>hi)).sum()
z = (s - s.mean())/s.std(ddof=0) # regle z : |z|>3, SEULEMENT si normal
```

Ensuite classer chaque outlier : **impossible** (age negatif) → corriger/NaN; **extreme mais reel** (gros client) → garder, ou "caper" (winsoriser) pour un modele lineaire. **Jamais** tout jeter.

Asymetrie & echantillonnage

Asymetrie a droite ⇒ moyenne > mediane ⇒ la moyenne surestime le client typique → reporter la mediane.

Variabilite d'echantillonnage : deux echantillons donnent des stats differentes. **Erreur standard de la moyenne** = $\sigma/\sqrt{n}$ → retrecit quand n grandit (TCL : la loi de la moyenne tend vers une normale quel que soit le fond).

Correlation

```
df.corr() (Pearson = lineaire);
df.corr(method="spearman") (rangs, monotone, robuste). |r| proche de 0 ⇒ pas de lien lineaire (peut etre non lineaire). |r| > 0,85 entre 2 features = colinearite : nuit a la stabilite des coefficients d'un modele lineaire, sans effet sur les arbres.
```

5. Probabilites

$$P(A \cup B) = P(A) + P(B) - P(A \cap B)$$

$$P(A \cap B) = P(A) P(B | A) \quad (\text{chaine})$$

$$P(A \cap B) = P(A) P(B) \iff A, B \text{ independants}$$

$$P(A | B) = \frac{P(B | A) P(A)}{P(B | A) P(A) + P(B | \neg A) P(\neg A)}$$

```
def bayes(p_a, p_b_a, p_b_not_a):
    p_b = p_b_a*p_a + p_b_not_a*(1-p_a)
    return p_b_a*p_a / p_b
```

Independance : A, B independants $\iff P(A | B) = P(A) \iff P(A \cap B) = P(A)P(B)$.

Negligence du taux de base (piege classique) : si $P(A)$ est minuscule, meme un test tres bon donne $P(A | +)$ faible, car les faux positifs du grand groupe " $\neg A$ " ecrasent les vrais positifs.

Ex. test medical : prevalence 0,5%, sensibilite 99%, specificite 95% $\Rightarrow P(\text{malade} | +) \approx 0,09$.

Juger une regle d'alerte pour une campagne : comparer $P(\text{classe} | \text{alerte})$ au taux de base = le *lift* ; regarder aussi $P(\text{classe} | \neg \text{alerte})$ (ce qu'on rate).

Piege : confondre $P(+ | \text{malade})$ et $P(\text{malade} | +)$; oublier le terme faux positif au denominateur.

6. ML – fondamentaux

split : quand	d'abord ; ajuster tout transformer sur le train seul
validation	tranche mise de cote pour le reglage, test intact
CV k-fold	fait tourner le fold de test k fois ; moyenne = estimation stable
surapprentissage	top au train, mauvais au test → regulariser / + de donnees / features + simples
sous-apprentissage	mauvais aux deux → + de capacite / meilleures features
biais	erreur d'un modele trop simple
variance	erreur de sensibilite a l'echantillon (modele trop flexible)
L1 (Lasso)	met des coefficients exactement a 0 (selection)
L2 (Ridge)	retrecit tous les coefficients, stabilise la colinearite
fuite (leakage)	info du futur / derivee de la cible / test dans le fit
scaling	requis kNN/SVM/logistique/NN/PCA ; inutile arbres/forets/boosting
one-hot	categorie nominale, faible cardinalite
ordinal	seulement si ordre reel (low<med<high)
target encoding	haute cardinalite, mais a ajuster dans les folds CV
desequilibre	accuracy $\approx$ 1-prevalence pour "predire la majorite" → recall/F1/ROC-AUC/PR-AUC ; class_weight , reechantillonnage, seuil

Diagnostic rapide : ecart train $\gg$ test = surapprentissage

sage; les deux bas = sous-apprentissage; "trop beau" = suspecter une fuite.

3 fuites typiques churn : colonne post-evenement (`date_annulation`); `StandardScaler`/encodeur ajuste sur toutes les lignes avant le split; feature calculee par un `groupby.transform("sum")` sur tout l'historique (fuite temporelle); clients dupliques a cheval sur train/test.

7. Metriques de classification

	predit +	predit -
reel +	TP	FN
reel -	FP	TN

precision = $TP/(TP+FP)$ – "parmi les signales, combien justes" (cout d'une fausse alerte)

recall = $TP/(TP+FN)$ – "parmi les vrais positifs, combien attrapes" (cout d'un oubli)

F1 = $2PR/(P+R)$ – un seul nombre quand les deux comptent

accuracy = $(TP+TN)/total$ – inutile si desequilibre

ROC-AUC : proba qu'un positif tire au hasard soit mieux classe qu'un negatif; sans seuil; *necessite predict_proba*.
PR-AUC (`average_precision`) : meilleur que ROC-AUC en fort desequilibre.

Choix de metrique :

- fraude / maladie / churn a retenir (un oubli coute cher) → **recall** (+ PR-AUC)
- spam / "ne pas deranger les bons users" → **precision**
- "appeler le top k" / classement → **ROC-AUC** ou **precision@k**
- classes equilibrees, couts symetriques → accuracy convient
- regression → MAE (robuste), RMSE (penalise les gros ecart), R^2

```
precision@k      :      thr = np.quantile(proba, 1-k);
precision_score(y, proba >= thr).
```

8. ML – squelette sklearn (par coeur)

```
from sklearn.model_selection import train_test_split,
    cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score,
    recall_score,
    f1_score, roc_auc_score, average_precision_score,
    confusion_matrix)

X = df.drop(columns=[TARGET, "customer_id"])
y = df[TARGET]
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42) #
    stratify TOUJOURS

ct = ColumnTransformer([
    ("num", StandardScaler(), NUM),
    ("cat", OneHotEncoder(handle_unknown="ignore"), CAT)])

pipe = Pipeline([("prep", ct),
    ("clf", LogisticRegression(max_iter=1000, class_weight="
    balanced"))])
pipe.fit(X_tr, y_tr)
pred = pipe.predict(X_te)
proba = pipe.predict_proba(X_te)[:, 1] # AUC exige proba,
    pas pred

roc_auc_score(y_te, proba)
average_precision_score(y_te, proba)
confusion_matrix(y_te, pred)
cross_val_score(pipe, X, y, cv=5, scoring="roc_auc").mean() #
    sur le PIPE
```

```
# coefficients signes (direction) :
names = NUM + list(pipe.named_steps["prep"]
    .named_transformers_["cat"].get_feature_names_out(CAT))
pd.Series(pipe.named_steps["clf"].coef_[0], index=names) \
    .sort_values(key=abs, ascending=False).head(10)
```

Toujours : **stratify=y**; transformers *dans* le Pipeline (pas de fuite en CV); ROC-AUC depuis **predict_proba**; **handle_unknown="ignore"**; ne pas titrer sur l'accuracy en desequilibre; **class_weight="balanced"** pour que le modele ne predise pas que la classe majoritaire.

```
cross_val_score scoring : "accuracy" "precision"
"recall" "f1" "roc_auc" "average_precision"
"neg_mean_absolute_error" "r2".
```

9. Reconnaître le type de question (10 s)

Indices dans l'enonce	Reflexe
"diagnostiquer / decrire le dataset"	scan 6 lignes + liste des types faux
"total X par Y ou Z"	<code>df[masque].groupby(Y)[X].sum().sort_v</code>
"top N ... apres exclusion des invalides"	squelette mini-challenge (sec. 3)
"taux / part / % de ..."	<code>groupby(k)[boolean].mean()</code>
"par mois / semaine / jour"	<code>groupby(s.dt.to_period("M"))[v].agg(</code>
"depuis les N derniers jours"	<code>cutoff = d.max()-Nd</code> ; agreger la fenetre; <i>left-join</i> sur la liste complete des entites; <code>fillna(0)</code>
"les clients qui ont plus de K ..."	condition <i>par entite</i> : <code>groupby(id).agg</code> → set d'ids → filtrer les faits
"P(cause preuve)"	Bayes + probabilite totale; juger par le lift
"X et Y sont-ils independants?"	comparer $P(Y X)$ a $P(Y)$
"construire un classifieur + choisir la metrique"	<code>split(stratify)</code> → Pipeline(<code>ColumnTransformer</code> , <code>clf</code>) → 5 metriques → argumenter selon cout FP/FN + equilibre
"accuracy elevee mais recall faible"	desequilibre + mauvaise metrique
"trouver la source de fuite"	fit-avant-split / feature du futur / transform sur tout l'historique
"expliquer / interpreter le modele"	coefficients signes (logistique) ou <code>feature_importances_</code> (foret) + phrase metier

10. Checklist finale (cocher = su sans doc)

Cleaning : scan 6 lignes; `to_numeric/to_datetime` coerce; normaliser categories; impossible → NaN → mediane; fill par groupe (**transform**); doublons exacts + cle metier + **assert**.

Querying : filtre multi-conditions (`&`, `isin`, `between`); `.query`; `groupby.agg(nomme=...)`; taux = moyenne d'un boolean; top-N en une chaine; `left` vs `inner` + **indicator**; `.dt` + `to_period`; table 1-ligne-par-entite + **assert**.

Stats/Proba : mediane vs moyenne + pourquoi; bornes IQR; erreur standard $\sigma/\sqrt{n}$; Bayes avec denominateur complet; independance $P(Y | X)$ vs $P(Y)$; negligence du

taux de base.

ML : `split(stratify)` avant tout `fit`;
`Pipeline(ColumnTransformer, clf)` de memoire; one-hot
`handle_unknown="ignore"`; scaling (et arbres = non);
5 metriques + matrice de confusion; `cross_val_score`
sur le pipe; `precision@k` via `np.quantile`; coefficients
via `get_feature_names_out`; surapprentissage vs sous-
apprentissage a partir de l'ecart train/test; 3 sources de
fuite; choix de metrique selon cout FP/FN.

Vitesse : mini-challenge Day 1 ≤ 25 min; challenge Day
2 ≤ 30 min; mock Day 3 : 10 questions atteintes en ~ 90
min, score > 75 .

*Regle du test : Python uniquement, 90 min, pas d'IA, doc de syn-
taxe seulement si vos regles d'entrainement l'autorisent. Budgeter le
temps par question; une reponse suffisante atteinte vaut mieux qu'une
reponse parfaite jamais rendue.*